Aim

To figure out why this Selection Sort code is sorting things backwards, fix the actual bug, and then tighten up the code a bit so it doesn't do extra work it doesn't need to.

Problem Statement

We're given a Selection Sort function that's *supposed* to sort a list in ascending order (smallest to largest). But something's off — instead it comes out in descending order. We need to dig into the comparison logic, figure out exactly where it's going wrong, fix it, confirm the fix actually works, and then clean up the swapping so we're not doing pointless swaps.

Step 1: Original (Buggy) Code

```
1  def selection_sort(arr):
2      n = len(arr)
3      for i in range(n):
4          min_idx = i
5          for j in range(i+1, n):
6              if arr[j] > arr[min_idx]:
7                  min_idx = j
8          arr[i], arr[min_idx] = arr[min_idx], arr[i]
9      return arr
10
11 data = [5, 2, 9, 1, 5]
12 print(selection_sort(data))
```

Go ahead and run this. You'll get:

```
Output
[9, 5, 5, 2, 1]
```

That's backwards — it's sorted largest to smallest, when we wanted the opposite.

Step 2: What's Actually Going Wrong:

- **The check arr[j] > arr[min_idx] looks for the larger value, not smaller.**
- **So min_idx ends up pointing to the maximum, even though it's named "min."**
- **Biggest value gets pushed to front each pass → whole array ends up descending.**
- **Fix: change > to <.**

Step 3: Fixed Code

main.py

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:    # fixed
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

data = [5, 2, 9, 1, 5]
print(selection_sort(data))
```

Output:

Output

```
[1, 2, 5, 5, 9]

=== Code Execution Successful ===
```

Correct — ascending order. Bug fixed.

Step 4: Optimization Idea

- Code swaps on every pass, even when the element is already correctly placed.
- That's a wasted swap (swapping a value with itself).
- Fix: only swap if min_idx != i.

Step 5: Final Optimized Code

main.py

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:                # skip needless swap
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

data = [5, 2, 9, 1, 5]
print(selection_sort(data))
```

Output

```
[1, 2, 5, 5, 9]

=== Code Execution Successful ===
```

Step 6: Complexity

- Comparisons: always n(n-1)/2, regardless of input order.
- Time Complexity: $O(n^2)$ — best, average, worst case all same.
- Space Complexity: O(1) — sorts in place, only uses min_idx.
- Swap optimization saves a few operations but doesn't change the overall $O(n^2)$ speed class.